

Problem Set 5

Due DateOctober 08, 2024 (11:59 PM)
Name **Hyma Jujjuru**
Student ID **110743269**
Collaborators

Contents

Instructions	1
Honor Code (Make Sure to Virtually Sign) [10 pts]	2
1 Recurrence Relations	3
1.1 Problem 1 [15 pts]	3
1.2 Problem 2 [30 pts]	5
2 Mergesort	7
2.1 Problem 3 [15 pts]	7
2.2 Problem 4 [30 pts]	8

Instructions

- The solutions **should be typed**, using proper mathematical notation. We cannot accept hand-written solutions. Here's a short intro to $\text{\LaTeX}$.
- You should submit your work through the **class Gradescope page** only (linked from Canvas). Please submit one PDF file, compiled using this $\text{\LaTeX}$ template.
- You may not need a full page for your solutions; pagebreaks are there to help Gradescope automatically find where each problem is. Even if you do not attempt every problem, please submit this document with no fewer pages than the blank template (or Gradescope has issues with it).
- You are welcome and encouraged to collaborate with your classmates, as well as consult outside resources. You must **cite your sources in this document**. **Copying from any source is an Honor Code violation. Furthermore, all submissions must be in your own words and reflect your understanding of the material.** If there is any confusion about this policy, it is your responsibility to clarify before the due date.
- Posting to **any** service including, but not limited to Chegg, Reddit, StackExchange, etc., for help on an assignment is a violation of the Honor Code.

- You **must** virtually sign the Honor Code (see Section). Failure to do so will result in your assignment not being graded.

Honor Code (Make Sure to Virtually Sign) [10 pts]

Problem HC. • My submission is in my own words and reflects my understanding of the material.

- Any collaborations and external sources have been clearly cited in this document.
- I have not posted to external services including, but not limited to Chegg, Reddit, StackExchange, etc.
- I have neither copied nor provided others solutions they can copy.

Agreed (signature here). I agree to the above, Hyma Jujjuru.

□

1 Recurrence Relations

1.1 Problem 1 [15 pts]

Problem 1. Consider the following recursive codes:

1. Obtain the recurrence relation for the given algorithms.
2. Solve the recurrence relation to get the run time complexity of the algorithm in the form $T(n) = \Theta(f(n))$

```
procedure FUNC( $n$ )  
  if  $n \leq 1$  then  
    return 1  
  for  $i \leftarrow 1$  to  $\sqrt{n}$  do  
    print "Algorithms!"  
   $left\_result \leftarrow \text{FUNC}(n/4)$   
   $right\_result \leftarrow \text{FUNC}(n/4)$   
  return  $left\_result + right\_result + \sqrt{n}$ 
```

```
procedure RECURSION( $n$ )  
  if  $n \leq 1$  then  
    return 1  
   $left\_result \leftarrow \text{RECURSION}(n - 1)$   
   $right\_result \leftarrow \text{RECURSION}(n - 1)$   
  return  $left\_result + right\_result$ 
```

Proof. Show the recurrence relation and solve for the run time complexity in the form $T(n) = \Theta(f(n))$.
 $FUNC(n)$

$$T(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ 2T(\frac{n}{4}) + \Theta(\sqrt{n}) & \text{if } n > 1 \end{cases} \quad (1)$$

$$\Theta(\sqrt{n}) = c\sqrt{n}$$

Note: $4^k = n, k = \log_4 n \rightarrow$ there are $\log_4 n$ levels to reach the base case where k is the number of times the recursion is done.

$$d = c+1$$

$$\begin{aligned}
T(n) &= 2T\left(\frac{n}{4}\right) + c\sqrt{n} \\
&= 2\left[2T\left(\frac{n}{4^2}\right) + c\sqrt{\frac{n}{4}}\right] + c\sqrt{n} = 2^2T\left(\frac{n}{16}\right) + 2c\sqrt{n} \\
&= 2^2\left[2T\left(\frac{n}{4^3}\right) + c\sqrt{\frac{n}{16}}\right] + 2c\sqrt{n} = 2^3T\left(\frac{n}{4^3}\right) + 3c\sqrt{n} \\
&= \dots \\
&= 2^iT\left(\frac{n}{4^i}\right) + ic\sqrt{n} \\
&= 2^{\frac{\log_2 n}{\log_2 2}}T\left(\frac{n}{4^{\log_4 n}}\right) + \log_4 n \cdot c\sqrt{n} \\
&= 2^{\frac{\log_2 n}{2}}T\left(\frac{n}{n}\right) + \log_4 n \cdot c\sqrt{n} \\
&= 2^{\log_2 n \cdot \frac{1}{2}}T\left(\frac{n}{n}\right) + \log_4 n \cdot c\sqrt{n} \\
&= n^{\frac{1}{2}}T(1) + \log_4 n \cdot c\sqrt{n} \\
&= \sqrt{n}(1) + \log_4 n \cdot c\sqrt{n} \\
&= \sqrt{n} + \log_4 n \cdot c\sqrt{n} \\
&= \sqrt{n} + c\sqrt{n} \log_4 n
\end{aligned}$$

Run Time Complexity: Since n is the value with the highest order, $T(n) = \Theta(\sqrt{n} \log n)$.
RECURSION(n)

$$\begin{aligned}
T(n) &= \begin{cases} 1 & \text{if } n \leq 1 \\ 2T(n-1) + \Theta(1) & \text{if } n > 1 \end{cases} \quad (2) \\
\Theta(1) &= c \\
\text{Note: } n-i=1 &\longrightarrow i=n-1
\end{aligned}$$

$$\begin{aligned}
T(n) &= 2T(n-1) + c \\
&= 2[2T(n-2) + c] + c = 2^2T(n-2) + 3c \\
&= 2^2[2T(n-3) + c] + 3c = 2^3T(n-3) + 7c \\
&= \dots \\
&= 2^iT(n-i) + (2^i - 1)c \\
&= 2^{n-1}T(n-n+1) + (2^{n-1} - 1)c \\
&= 2^{n-1}T(1) + c \cdot 2^{n-1} - c \\
&= 2^{n-1} + c \cdot 2^{n-1} - c \\
&= c \cdot 2^{n-1} - c
\end{aligned}$$

Run Time Complexity: $T(n) = \Theta(2^n)$

□

1.2 Problem 2 [30 pts]

Problem 2. Imagine you're scrolling through your camera roll, trying to find that one perfect selfie you took a month ago. Instead of swiping through hundreds of photos, you can jump to the middle of the roll. You can remember if the selfie you're looking for was taken before this middle point, so you only need to check the earlier photos. If it's after, you focus on the later ones. Each time, you cut the number of photos you need to scroll through in half, until you find that perfect shot.

This is exactly how binary search works: it checks the midpoint of a sorted array (in this case, your photo roll) and eliminates half of the options from further consideration, repeating this process until the target is found.

Observe that the binary search algorithm repeats this procedure, halving the size of the remaining portion of the subarray each time.

- Write recursive pseudocode for binary search and give a small example (using a sorted array of 5 elements).
- Argue that the worst-case running time of binary search is $O(\log n)$.

Answer. Binary Search Pseudocode:

```
1 binary_search(L, left, right, ele)
2   if right < left
3       break when x does not exist
4   set mid = left + (right - left) // 2
5   if L[mid] < ele
6       binary_search(L, left + 1, right, ele)
7   if L[mid] > ele
8       binary_search(L, left, right - 1, ele)
9   if L[mid] = ele
10      return mid
```

Example: $L = [P1, P2, P3, P4, P5]$

`binary_search(L, 1, 5, P4):`

left = 1 right = 5 mid = $1 + (5-1)//2 = 1 + 2 = 3$

$L[mid] = L[3] = P3$ $P4 > P3 \implies \text{left} = 1 + 1 = 2$

`binary_search(L, 2, 5, P4):`

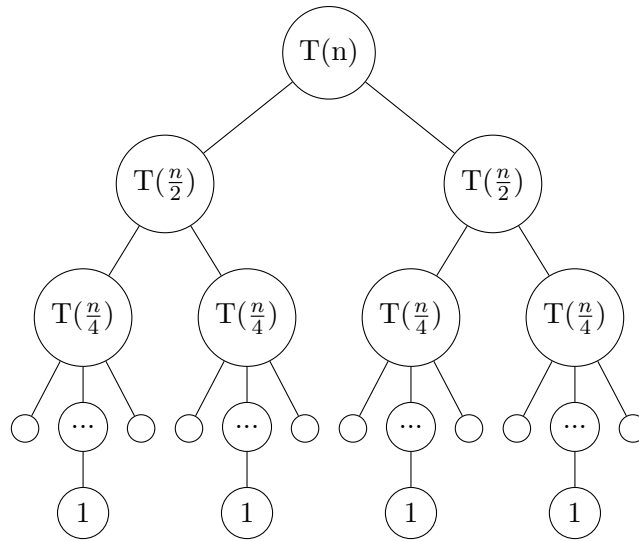
left = 1 right = 6 mid = $2 + (5-2)//2 = 2 + 2 = 4$

$L[mid] = L[4] = P4$

return location of photo, mid = 4

Time Complexity:

$$T(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ 2T(\frac{n}{2}) + \Theta(1) & \text{if } n > 1 \end{cases} \quad (3)$$



The worst case of binary search is that the target is at the first index or the last index. Since the algorithm halves the list each time, it takes a maximum of $\log_2 n$ recursions to reach the first index or the last index. So, the worst run time complexity of binary search is $O(\log n)$.

□

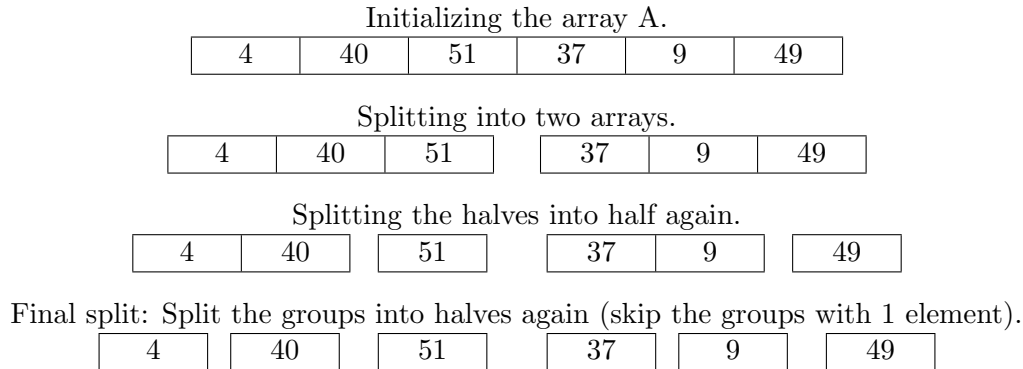
2 Mergesort

2.1 Problem 3 [15 pts]

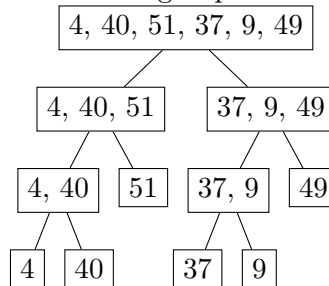
Problem 3. Using the mergesort algorithm, demonstrate how divide-and-conquer works on the array $A = [4, 40, 51, 37, 9, 49]$. Clearly explain the recursive steps and detail the merge operations at each stage.

Hint: Show how the array is split at each level and how the sorted subarrays are combined back together. Include diagrams or intermediate steps to illustrate the process.

Answer. Mergesort Algorithm: Divide the given array by two recursively each time until we reach subgroups of 1 element each. We merge the subgroups back into one while sorting the elements.

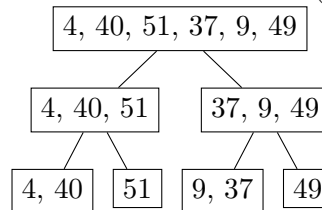


Let's label the groups into a tree.

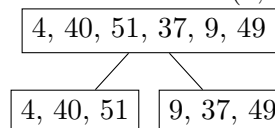


Now, we will start merging the elements back into one tree while sorting into ascending order.

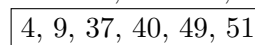
First merge the values in the 4th level: $40 > 4 = (4, 40)$ and $37 > 9 = (9, 37)$



Merge the values of the 3rd level: $51 > 4$ and $51 > 40 = (4, 40, 51)$ $49 > 9$ and $49 > 37 = (9, 37, 49)$



Merge the values of 2nd level: $9 > 4, 40 > 37, 51 > 49$ $(4, 9, 37, 40, 49, 51)$



□

2.2 Problem 4 [30 pts]

Problem 4. Recall the Mergesort algorithm, You are given k sorted arrays, each containing n elements. The goal is to merge them into one sorted array containing kn elements. We will explore two methods to achieve this and analyze their time complexities:

1. Sequential Merge Strategy:

- (a) **Merging Two Arrays:** To begin, consider merging two sorted arrays using a standard merge procedure (like in merge sort), which takes linear time based on the total size of the arrays. **Hint:** What is the time complexity of merging two arrays of size n ?
- (b) **Merging Multiple Arrays Sequentially:** Next, merge the first two arrays. After obtaining the merged result, merge the third array into it, and continue this process until all arrays are merged. **Hint:** After merging the first two arrays (size $2n$), the next merge involves $3n$ elements. How does the size increase after each step?

2. Using Divide-and-Conquer:

- (a) **Divide the Arrays:** Rather than merging arrays one by one, consider dividing the k arrays into pairs and merging them simultaneously. **Hint:** This divide-and-conquer strategy is similar to mergesort. If you merge half the arrays at each level, how many levels will there be?
 - (b) **Merge Two Arrays at Each Level:** At each level, merge two arrays (or merged subarrays) of size n from the previous step. The total number of elements at each level remains kn , but how many levels are there? **Hint:** The number of levels corresponds to $\log k$.
3. Which approach is more efficient, and why? Think about how divide-and-conquer helps reduce the time complexity when handling large datasets.

Answer. Two Sorting Methods:

1. Sequential Merge Strategy:

- (a) **Merging Two Arrays:**
The time complexity of merging two arrays of size n in linear time is $O(n + n) = O(2n)$.
- (b) **Merging Multiple Arrays Sequentially:**
Process: Merging two arrays then merging that with a third array and continuing this until all k arrays are merged.
Ex. two arrays = $O(2n)$, three arrays = $O(2n + n = 3n)$, four arrays = $O(3n + n = 4n)$, ...
The size increases by n for each array added. So the time complexity is $O(kn)$

2. Using Divide-and-Conquer:

- (a) **Divide the Arrays:**
Process: divide each of the k arrays into pairs and merge them simultaneously.
You divide the arrays into halves until you reach the base case, which is 1 element in this case, so there will be $\log_2 k$ levels.
- (b) **Merge Two Arrays at Each Level:**
Merging two subarrays of size n with the arrays from the previous step is similar to mergesort where the number of elements at each level is kn and the number of levels is $\log_2 k$. So the time complexity is $O(kn \log k)$.

Divide-and-Conquer is the better option because its time complexity is $O(kn \log k)$ which is a better run time complexity than $O(kn)$. In other words, using divide and conquer and merging back allows the arrays to be merged and sorted simultaneously instead of sequentially which takes more time. $\square$